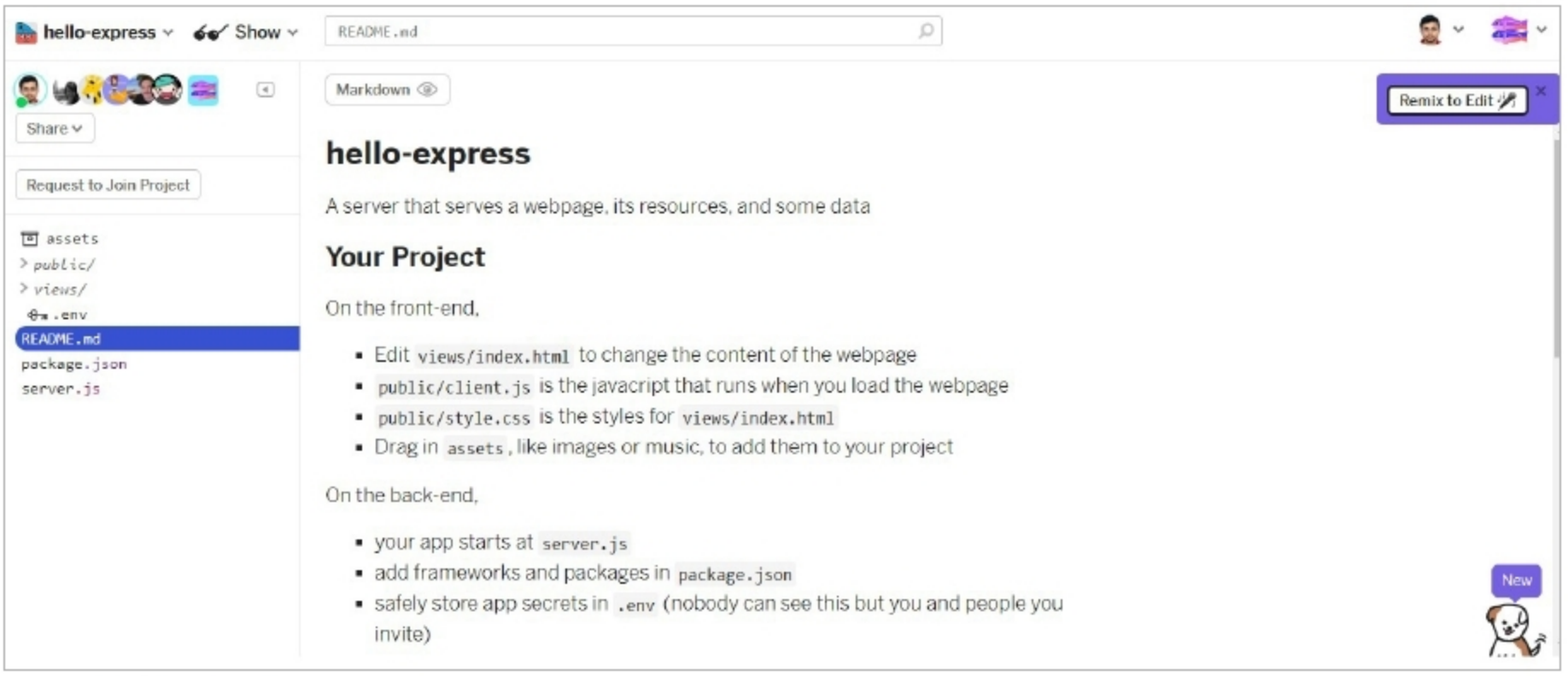

Figure 1: Glitch GUI for hello-express

make use of the latter and enhance your own application if you have one, or get inspired enough to write your first app as I was, just to get some experience with PWAs.

Note: I took lots of help and some boilerplate code from a Google Code Labs article that, though good for a start, is still incomplete with regard to many instructions, which I have tried to improve upon in this article.

Setting up a coronavirus data scraper in Glitch

A quick introduction to Glitch and hello-express: Glitch is an all-in-one tool to create and deploy Web apps from the ground up. It has lots of collaborative features that allow multiple people to work on a project, and it also provides different startup projects with which you can mix and create your own version to progress faster. For our purposes, let's start with the hello-express project that has a simple Web app, which serves a list of dreams and allows you to add your own dreams to the list. For this article, we will replace dreams with corona statistics, basically showing the total worldwide cases and deaths.

Note: 1. Due to the immediate deployment of code into your server, Glitch facilitates CI/CD (continuous integration/continuous deployment) such that any changes in your code can be tested continuously.

2. As a corollary to the above, it makes lots of sense to keep your code running all the time, so you know if you have broken something and can fix it immediately. Also, Glitch has some more great features like live bug reporting in code and some neat code beautification tools too.

3. Before you start really mixing and coding, you need to register yourself in Glitch to create your account, activate it, etc. Setting up is standard and you can use your Google sign-in to get started.

Package.json

Let us start with package.json, which instructs Glitch about what packages are needed for our project and indicates our dependencies. As you can see in the code below, it is indicated in the dependencies section that the Express framework is needed. As we will need to do some Web scraping, let us add the Cheerio and Request packages also. Do note that in the *Start* section, the node is started with the *Server.js* script file.

```
{
  "//1": "describes your app and its
dependencies",
  "//2": "https://docs.npmjs.com/files/
package.json",
  "//3": "updating this file will
download and update your packages",
  "name": "hello-express",
  "version": "0.0.1",
  "description": "A simple Node app
built on Express, instantly up and
running.",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.17.1",
  },
  "engines": {
    "node": "12.x"
  },
  "repository": {
    "url": "https://glitch.com/edit/#!/
hello-express"
  },
  "license": "MIT",
```